Rust製 常駐メモリ最適化ツール 設計書（Windows 10/11）

Nov 12, 2026 · @YUTAKA

1. 概要と設計方針

本ツールは「メモリ不足の原因を見せ、必要なときだけ安全に解放する」常駐アプリです。固定間隔で強制解放するのではなく、メモリ圧迫を検知したときに段階的に解放し、効果が悪ければ自動で控えめにします。

主な機能

1. **常駐・定期解放**: Windowsサービスとして常駐し、メモリ圧迫・アイドル状態・スケジュールを条件に解放処理を実行する。
2. **消費元の可視化**: プロセス単位ではなく「機能」単位（アプリ、ブラウザ、Windowsサービス、WSL、スタートアップ項目など）にまとめてメモリ消費を表示する。
3. **機能の停止**: 表示された機能を、安全分類に応じた方法（正常終了、サービス停止、スタートアップ無効化など）で停止し、後から元に戻せる。

設計原則

- **既定は安全側**: 初期状態で有効なのは軽い解放（バックグラウンドのワーキングセット整理と低優先度スタンバイ解放）のみ。スタンバイ全消去は手動か明示設定時だけ。
- **見せてから止める**: 停止は必ずユーザー操作から始まる。自動で他プロセスを終了させる機能は持たない。
- **すべて元に戻せる**: 永続的な変更（サービスのスタートアップ種別、スタートアップ無効化など）は記録し、ワンクリックで復元できる。
- **効果を測る**: 解放前後の空きメモリだけでなく、その後のハードページフォルトとディスク読み込みも記録し、逆効果を検出する。
- **最小権限**: GUIは一般ユーザー権限で動かし、特権が必要な処理だけをサービスに任せる。

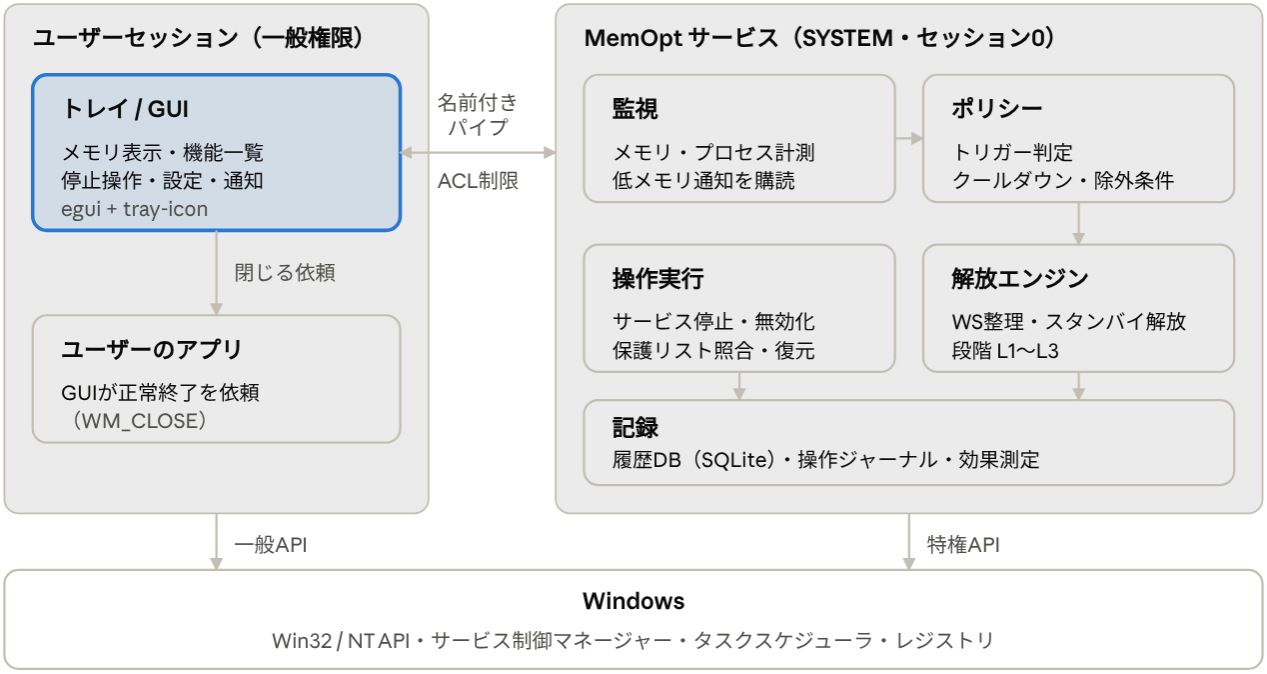
非目標

- 「空きメモリを常に最大化する」こと。Windowsのスタンバイメモリは有用なキャッシュであり、空にすると遅くなる場合がある。
- カーネルドライバの導入。すべてユーザーモードの公開・準公開APIで実装する。
- ウイルス対策製品や保護プロセス（PPL）への干渉。

2. 全体アーキテクチャ

特権が必要な解放・停止処理は常駐サービスに、表示と操作はユーザー権限のトレイGUIに分けます。GUIを管理者やSYSTEMで動かさないため、GUIの脆弱性が全体の権限昇格につながりません。

特権処理はサービス、操作はユーザー側GUIに分ける



全体構成 · 2プロセス + 名前付きパイプ

GUIが落ちても常駐解放は続き、サービスが停止中でもGUIは読み取り専用の監視モードで動きます。

プロセスの役割

コンポーネント	実行権限	責務
memopt-service	LocalSystem（Windowsサービス、自動起動）	監視、トリガー判定、解放、サービス停止などの特権操作、履歴記録
memopt-tray	ログオンユーザー（一般権限、スタートアップ登録）	トレイアイコン、ダッシュボード、機能一覧、停止の確認ダイアログ、通知、ウィンドウを持つアプリの正常終了
memopt-cli（任意）	管理者	手動解放、診断ダンプ、サービスのインストール／削除

重要な分担: アプリの「正常終了」はGUI側で行う

サービスはセッション0で動くため、ユーザーのデスクトップ上のウィンドウに `WM_CLOSE` を送れません。保存確認ダイアログを出させる正常終了はGUIが行い、サービスは強制終了（`TerminateProcess`）とサービス停止だけを担当します。

IPC（名前付きパイプ）

- パイプ名 `\\.\pipe\memopt-v1`。セキュリティ記述子で「対話ユーザーとAdministrators」のみ接続可に制限する。
- 1行1メッセージのJSON（`serde`）。全メッセージに `protocol_version` を付け、GUIとサービスの版ずれを検出する。
- 要求例: `GetSnapshot`, `GetFeatures`, `RunClean{level}`, `StopFeature{id, method}`, `Undo{journal_id}`, `UpdateConfig`。
- サービスからの通知: `CleanFinished`, `PressureChanged`, `ActionNeedsConfirmation`。
- 破壊的な要求（停止・永続変更）はサービス側でも保護リストを再照合する。GUIの判定だけを信用しない。

MVPの簡易構成

最初はサービスを作らず、タスクスケジューラの「ログオン時・最上位の特権で実行」でGUI単体を管理者起動してもよいです。ただしUAC昇格したプロセスでGUIを動かすことになるため、正式版では上記の2プロセス構成へ移行します。

3. 常駐と定期メモリ解放

解放は「固定間隔で毎回実行」ではなく、一定間隔で状態を評価し、条件を満たしたときだけ最も軽いレベルから実行します。評価間隔は既定15秒、実際の解放は最短でも5分おき（クールダウン）です。

解放レベル

レベル	内容	主なAPI	必要な権限	副作用	既定
L1 軽	非アクティブなバックグラウンドプロセスのワーキングセット整理	`SetProcessWorkingSetSizeEx(h, -1, -1)`（= `EmptyWorkingSet`）	対象プロセスへの `PROCESS_SET_QUOTA`	再利用時のソフトフォルト増加（ページはスタンバイ/変更リストに移るだけ）	自動で有効
L2 中	低優先度スタンバイリストの解放	`NtSetSystemInformation(SystemMemoryListInformation, MemoryPurgeLowPriorityStandbyList)`	`SeProfileSingleProcessPrivilege`	先読み（SysMain）キャッシュの一部が失われる	自動で有効

レベル	内容	主なAPI	必要な権限	副作用	既定
L3 強	スタンバイリスト全体の解放＋変更ページのフラッシュ	同 `MemoryPurgeStandbyList` / `MemoryFlushModifiedList`	同上	ファイルの再読み込み増加、一時的なディスクI/O増	手動のみ（設定で自動化可）
特殊	システムファイルキャッシュの縮小	`SetSystemFileCacheSize(-1, -1, 0)`	`SeIncreaseQuotaPrivilege`	L3と同様	手動のみ

- L1の対象は「最前面でない」「直近N秒CPU使用率が低い」「保護リスト外」のプロセスに限る。全プロセス一括の `MemoryEmptyWorkingSets` は使わない（最前面アプリまで遅くなるため）。
- サービス起動時に `AdjustTokenPrivileges` で必要な特権を有効化し、失敗したレベルはGUIに「利用不可」と表示する。

トリガー

トリガー	条件（既定値）	実行するレベル
メモリ圧迫	利用可能メモリ < 15%、またはコミット済み / コミット上限 > 85% が30秒継続	L1 → 改善しなければL2
低メモリ通知	`CreateMemoryResourceNotification(LowMemoryResourceNotification)` がシグナル	L1 + L2
アイドル時	ユーザー無操作10分以上（`GetLastInputInfo` をGUIが中継）	L1のみ
スケジュール	任意（例: 毎日3:00）	設定したレベル
手動	トレイメニュー / ショートカットキー	選択したレベル

評価ループ

1. 15秒ごとにスナップショットを取得する（`GlobalMemoryStatusEx`、`GetPerformanceInfo`、PDHのスタンバイ・変更リスト・ハードフォルトのカウンター）。
2. トリガー条件を判定する。しきい値には上下の幅（ヒステリシス、例: 15%で発動・25%で解除）を設け、境界付近で連続発動しないようにする。
3. 抑止条件を確認する。該当すれば実行せず、理由を履歴に残す。
4. 最も軽いレベルを実行し、直後と60秒後の値を記録する。
5. 60秒後も条件が続いていれば1段階上げる。ただし自動で上がるのはL2まで。
6. クールダウンを開始する（既定5分、効果が低い場合は自動延長。7章参照）。

抑止条件（実行しない場面）

- 全画面アプリやゲーム、プレゼン中（`SHQueryUserNotificationState` が `QUNS_RUNNING_D3D_FULL_SCREEN` / `QUNS_PRESENTATION_MODE` / `QUNS_BUSY`）。判定はユーザーセッションのGUIが行い、サービスへ通知する。
- ディスク使用率が高い（例: 過去30秒の平均ディスク使用率 > 70%）。L3はさらにI/Oを増やすため。
- バッテリー駆動かつ省電力モード（設定で切替）。
- 直前の解放後にハードフォルトが大きく増えた（逆効果の判定。7章）。
- 起動後5分以内（起動直後の正常な読み込みを妨げないため）。

常駐自体の負荷目標

- 常駐時のワーキングセット: サービス 20MB以下、トレイ（ウィンドウ非表示時）30MB以下。
- CPU: 評価ループは平均0.1%未満。全プロセス列挙は5秒以上の間隔に限定し、GUIが開いている間だけ2秒更新にする。
- 自プロセスのワーキングセットも解放対象から除外せず、アイドル時に自分自身も整理する。

4. メモリ消費の可視化（機能単位）

プロセス名の一覧ではなく、ユーザーが止める判断をできる「機能」単位にまとめて表示します。例えばChromeの30個のプロセスは「Google Chrome」1行、svchost内の各サービスは「Windows Search」「SysMain」のようにサービス名で1行にします。

機能の種類と集約方法

種類	集約のしかた	識別に使う情報	停止手段（5章）
デスクトップアプリ	実行ファイルパスが同じプロセスと、その子プロセスを1つにまとめる	親PID（`CreateToolhelp32Snapshot`）、`QueryFullProcessImageNameW`、ファイルの製品名（`GetFileVersionInfoW`）	正常終了 → 強制終了
ブラウザ	上記に加え、Chromium系/Firefoxのレンダラー・GPU・ユーティリティを親ブラウザに集約	コマンドライン（`--type=renderer` など）	ブラウザ全体の正常終了、タブ整理の案内

種類	集約のしかた	識別に使う情報	停止手段（5章）
ストアアプリ（UWP/MSIX）	パッケージファミリー名でまとめる	`GetPackageFullName`	正常終了
Windowsサービス	1サービス = 1行。svchostを共有する場合はサービスグループで1行にし、内訳を展開表示	`EnumServicesStatusExW` のPID、表示名・説明	サービス停止 / スタートアップ種別変更
スタートアップ項目	登録元（Runキー、スタートアップフォルダー、タスク）ごとに、起動中のプロセスと結び付ける	`HKCU/HKLM\...\Run`、`StartupApproved`	次回から無効化
スケジュールタスク	実行中のタスクとそのプロセス	タスクスケジューラCOM（`ITaskService`）	タスク無効化
WSL・仮想マシン	`vmmemWSL` / `vmmem` を独立した機能として表示	プロセス名	`wsl --shutdown`、`.wslconfig` の上限設定を案内
システム	`Memory Compression`、カーネル（ページプール/非ページプール）、ドライバ	`GetPerformanceInfo`	停止不可（表示のみ）

表示する値と注意点

- **主指標はプライベートバイト（コミット）**。`PROCESS_MEMORY_COUNTERS_EX2.PrivateUsage` を合計する。共有DLLを重複して数えないため、機能単位の合計に向いている。
- **補助指標としてワーキングセット**（物理RAMに今載っている量）を並べて表示する。ワーキングセットは共有ページを含むため、合計値は表示しない。
- 「停止したら空く見込み」はプライベートワーキングセット（`PrivateWorkingSetSize`）の合計で示し、「見込み」と明記する。
- 増加傾向: 各機能の直近30分の推移を記録し、一定時間ごとに増え続けているものに「増加中」バッジを付ける（メモリリーク候補）。
- 取得できない値（保護プロセス、他ユーザーのプロセス）は空欄にし、理由をツールチップで示す。

画面構成

- **上部**: 物理メモリの内訳バー（使用中 / 変更済み / スタンバイ / 空き）と、コミット使用率。スタンバイは「再利用可能なキャッシュ」と説明を添える。
- **中央**: 機能一覧（使用量順）。列は 名前・種類・プライベート・ワーキングセット・推移ミニグラフ・安全分類・操作ボタン。
- **行の展開**: 構成プロセス、サービスの依存関係、起動元（スタートアップかどうか）を表示する。
- **下部**: 解放履歴（日時・レベル・解放量・60秒後の評価）。
- **トレイ**: アイコンに使用率を表示し、右クリックで「今すぐ軽く解放」「上位5機能」「一時停止（1時間）」を出す。

5. 機能の停止

停止は「今だけ止める」と「次回以降も止める」を明確に分け、後者はすべてジャーナルに記録して復元できるようにします。どの操作もユーザーの明示的な操作からのみ実行し、自動では行いません。

停止手段

手段	対象	実装	影響の範囲	復元
正常終了	ウィンドウを持つアプリ	GUIがトップレベルウィンドウに `WM_CLOSE` を送り、最大10秒待つ	今回のみ	ユーザーが再起動
強制終了	正常終了に応じないアプリ	サービスが `TerminateProcess`（プロセスツリー単位）	今回のみ。未保存データは失われる	ユーザーが再起動
サービス停止	Windowsサービス	`ControlService(SERVICE_CONTROL_STOP)`。先に `EnumDependentServicesW` で依存サービスを確認	今回のみ（再起動で戻る）	`StartServiceW`
スタートアップ種別変更	Windowsサービス	`ChangeServiceConfigW` で「手動」または「無効」	永続	元の種別をジャーナルから復元

手段	対象	実装	影響の範囲	復元
スタートアップ無効化	Runキー等の自動起動	タスクマネージャーと同じ `StartupApproved` の値を書き換え（登録自体は消さない）	次回ログオンから	値を元に戻す
タスク無効化	スケジュールタスク	`IRegisteredTask::put_Enabled(false)`	永続	再有効化
優先度を下げる	止めたくないが邪魔なアプリ	`SetProcessInformation(ProcessMemoryPriority, MEMORY_PRIORITY_LOW)`、必要なら効率モード	今回のみ	元に戻す / 終了で消える

- 「優先度を下げる」は停止の代替として必ず提示する。メモリ優先度を下げたプロセスのページは、圧迫時に先に手放されるため、アプリを残したまま他を優先できる。
- プロセスの一時停止（`NtSuspendProcess`）は、ウィンドウ応答なしやデッドロックの原因になるため採用しない。

安全分類

分類	例	GUIでの扱い
保護（停止不可）	System、Registry、smss、csrss、wininit、winlogon、services、lsass、Memory Compression、dwm、保護プロセス（PPL）、`SERVICE_ACCEPT_STOP` を持たないサービス、本ツール自身	操作ボタンを表示しない。理由をツールチップで示す
注意	explorer、audiodg、ネットワーク・音声・印刷関連サービス、ドライバ付属サービス、他ユーザーのプロセス、セッション0のプロセス	影響説明つきの確認ダイアログ。永続変更は2段階確認
停止可	一般のユーザーアプリ、ランチャー、アップデーター、同期クライアント	1回の確認で実行

分類は「組み込みルール → ユーザー設定 → 実行時の判定」の順で決め、最も厳しいものを採用します。実行時の判定には `ProcessProtectionLevelInfo`（PPL判定）、サービスの `dwControlsAccepted`、依存サービスの有無を使います。

停止の流れ

1. ユーザーが機能の行で「停止」を選ぶ。

2. GUIが影響を表示する: 解放見込み量、構成プロセス数、ウィンドウの有無、依存サービス、次回起動への影響。
3. 手段を選ばせる。既定は最も穏やかな手段（アプリ = 正常終了、サービス = 今回のみ停止）。
4. サービス側で保護リストを再照合し、許可されたものだけ実行する。
5. 結果と前後のメモリ量をジャーナルに記録し、通知に「元に戻す」ボタンを付ける。
6. 正常終了がタイムアウトした場合は、自動で強制終了せず、改めて確認する。

復元（Undo）

- 永続変更はすべて `journal.jsonl` に「操作前の値」とともに追記する（例: サービス名、元のスタートアップ種別）。
- GUIの「変更履歴」画面から個別に、または「すべての変更を元に戻す」で一括復元できる。
- アンインストール時は、ジャーナルに残る未復元の永続変更を一覧表示し、復元するか確認する。

6. 保護リストと安全制御

保護リストは解放（L1の対象選び）と停止（5章）の両方で共通に使い、照合はサービス側で必ず行います。

保護リストの3層

1. **組み込み（変更不可）**: 5章「保護」の分類。プロセスはフルパスと署名者で照合し、名前だけの一致では保護扱いにしない（偽装された同名プロセス対策）。
2. **ユーザー保護**: ユーザーが「このアプリは触らない」と指定したもの。L1のワーキングセット整理からも除外する（例: DAW、ゲーム、録画ソフト、仮想マシン）。
3. **自動保護**: 実行時の状態による一時的な除外。最前面ウィンドウのプロセス、オーディオ再生中のプロセス、直近60秒以内に起動したプロセス、全画面中のプロセス。

暴走を防ぐ制限

- 自動解放は1時間あたり最大6回。超えたら自動解放を止め、通知で原因（どのトリガーが連続しているか）を示す。
- 1回のL1で整理するプロセス数に上限（既定50）を設け、メモリ量の多い順に処理する。
- 停止操作は1件ずつ。複数機能の一括停止は提供しない。
- 各Windows API呼び出しの失敗（アクセス拒否など）は記録だけして次へ進み、リトライで繰り返し叩かない。

権限とセキュリティ

- サービスが有効化する特権は `SeProfileSingleProcessPrivilege`、`SeIncreaseQuotaPrivilege`、`SeDebugPrivilege`（他ユーザーのプロセス操作時のみ）に限り、それ以外は `AdjustTokenPrivileges` で外す。
- 設定ファイルとジャーナルは `%ProgramData%\MemOpt\` に置き、書き込みはSYSTEMとAdministratorsのみ。一般ユーザーによる設定変更はGUI経由（パイプ）だけにする。
- パイプ要求の内容（停止対象のPIDやサービス名）はサービス側で再検証し、PIDの再利用に備えて起動時刻も照合する。
- 実行ファイルにはコード署名を付ける。未署名の常駐・管理者権限ツールはSmartScreenや他のウイルス対策製品で警告されやすい。

フェイルセーフ

- 「緊急停止」：トレイメニューから自動解放を即時無効化できる。
- サービスがクラッシュした場合、SCMの回復設定で1回だけ再起動し、2回目以降は停止したままにする。
- 本ツールが加えた永続変更が原因でトラブルが起きても、セーフモードでジャーナルから復元できるCLIコマンドを用意する（`memopt-cli undo --all`）。

7. 効果測定とログ

各解放について「空いた量」と「その後に遅くなっていないか」を両方記録し、逆効果なら自動で解放を控えめにします。これが単純なメモリクリーナーとの最大の違いになります。

記録する値

タイミング	値	取得元
実行直前	利用可能・空き・スタンバイ・変更済みの各量、コミット使用率	`GlobalMemoryStatusEx`、PDH `\Memory\Standby Cache *`、`\Memory\Modified Page List Bytes`
実行直後	同上、処理時間、対象プロセス数、失敗数	同上
実行後60秒間	ハードフォルト率、ディスク読み込み量	PDH `\Memory\Pages Input/sec`、`\PhysicalDisk(_Total)\Disk Read Bytes/sec`
比較用	実行前60秒間の同じ値	常時記録しているリングバッファ

逆効果の判定と自動調整

- 実行後60秒のハードフォルト率が、実行前の2倍以上かつ一定量以上なら「逆効果」と判定する。
- 逆効果が続いた場合: 同じトリガーのクールダウンを倍にする（上限60分）。L2で起きた場合は、そのトリガーの自動実行をL1止まりにする。
- 効果が小さい場合（解放量 < 物理メモリの1%）: 次回の同じトリガーを1回スキップする。
- 調整内容はGUIの履歴に「なぜ控えめにしたか」として表示し、ユーザーが手動で戻せる。

保存先と保持期間

- `history.db`（SQLite）: スナップショット（1分粒度で7日、1時間粒度で90日）、解放履歴、機能別の使用量推移。
- `journal.jsonl`: 停止・永続変更の操作ジャーナル。削除しない（復元に必要なため）。
- `memopt.log`: 動作ログ（`tracing` + 日次ローテーション、14日保持）。
- 診断用に、直近の状態をまとめてZIP出力する機能を用意する（不具合報告用。個人のファイルパスは任意で伏せる）。

8. 設定ファイル仕様

設定は `%ProgramData%\MemOpt\config.toml` に置き、GUIから変更するとサービスが検証してから保存・即時反映します。不正な値は既定値に戻し、警告を出します。

```
schema_version = 1

[monitor]
eval_interval_sec   = 15
process_scan_sec    = 5      # GUI表示中は2秒
startup_grace_sec   = 300

[auto_clean]
enabled             = true
max_auto_level      = 2      # 1=WS整理, 2=低優先度スタンバイ, 3=全スタンバイ
cooldown_sec        = 300
max_runs_per_hour   = 6

[auto_clean.pressure]
available_below_pct = 15
release_above_pct   = 25     # ヒステリシス
commit_above_pct    = 85
sustain_sec         = 30
```

```
[auto_clean.idle]
enabled             = true
idle_min            = 10
level               = 1

[[auto_clean.schedule]]
cron  = "0 3 * * *"
level = 2

[suppress]
fullscreen          = true
disk_busy_pct       = 70
on_battery_saver    = true

[adaptive]
enabled             = true
hard_fault_ratio    = 2.0
max_cooldown_min    = 60

[l1]
max_processes       = 50
skip_foreground     = true
skip_audio          = true
min_idle_cpu_sec    = 30

[protect]
user_paths = [
  'C:\Program Files\OBS Studio\bin\64bit\obs64.exe',
]

[ui]
tray_shows_percent  = true
hotkey_clean        = "Ctrl+Alt+M"
notify_on_auto      = false
```

9. クレート構成と主要Windows API

Cargoワークスペースで、OSに依存しない判定ロジック（policy）とWindows API呼び出し（probe / cleaner / actions）を分けます。判定ロジックを純粋関数にしておくと、実機なしで単体テストできます。

ワークスペース構成

```
memopt/
├─ crates/
|  ├─ core/       型定義（Snapshot, Feature, Level, SafetyClass）、設定、IPCメッセージ
|  ├─ probe/      計測（メモリ状態、プロセス列挙、サービス・パッケージ情報、PDH）
|  ├─ grouping/   プロセス → 機能への集約（4章）
|  ├─ policy/     トリガー判定・抑止・クールダウン・自動調整（純粋関数）
|  ├─ cleaner/    解放レベルL1〜L3の実装、特権の有効化
|  ├─ actions/    停止・無効化・復元、保護リスト照合、ジャーナル
|  ├─ store/      SQLite履歴、ログ
|  └─ ipc/        名前付きパイプのサーバー/クライアント
├─ apps/
|  ├─ service/    Windowsサービス本体
|  ├─ tray/       トレイ + GUI
|  └─ cli/        管理・診断・復元用CLI
└─ tests/         VMでの結合テスト
```

主な依存クレート

用途	クレート	備考
Windows API全般	`windows`	必要な機能フラグだけ有効化（`Win32_System_ProcessStatus`、`Win32_System_Services` など）
NTネイティブAPI	`windows`（`Wdk_System_SystemInformation`）または `ntapi`	`NtSetSystemInformation` は文書化が限定的なため、ラッパーを1か所にまとめる
サービス化	`windows-service`	SCMへの登録・制御ハンドラ
非同期・タイマー	`tokio`	名前付きパイプ（`tokio::net::windows::named_pipe`）も同梱
GUI	`eframe` / `egui`	軽量。GPUIを選ぶ場合はGUIクレートだけ差し替える
トレイ	`tray-icon`	通知はWinRTのトースト通知
設定・IPC	`serde`、`toml`、`serde_json`	
履歴	`rusqlite`（bundled）	
ログ	`tracing`、`tracing-appender`	
ショートカット	`global-hotkey`	

`sysinfo` は手軽ですが、プライベートバイトや親子関係、svchost内のサービス対応など本設計に必要な値が足りないため、計測は `windows` クレートで直接実装する方針とします。

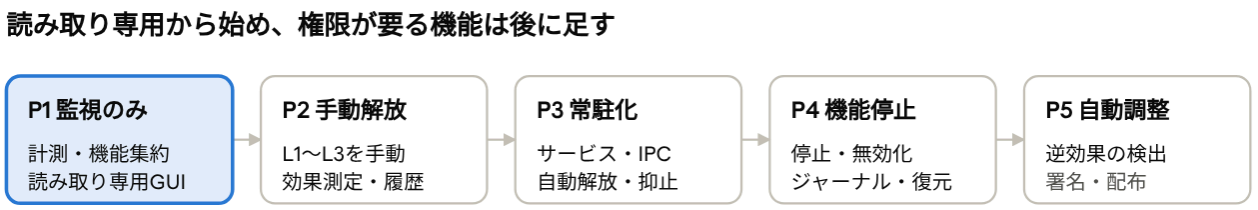
主要API一覧

処理	API
システム全体のメモリ	`GlobalMemoryStatusEx`、`GetPerformanceInfo`、PDH（`PdhAddEnglishCounterW`）
低メモリ通知	`CreateMemoryResourceNotification`、`QueryMemoryResourceNotification`
プロセス列挙・親子関係	`CreateToolhelp32Snapshot`、`Process32FirstW/NextW`
プロセスのメモリ量	`GetProcessMemoryInfo`（`PROCESS_MEMORY_COUNTERS_EX2`）
実行ファイル・パッケージ	`QueryFullProcessImageNameW`、`GetFileVersionInfoW`、`GetPackageFullName`
保護プロセス判定	`GetProcessInformation(ProcessProtectionLevelInfo)`
ワーキングセット整理	`SetProcessWorkingSetSizeEx`
スタンバイ解放	`NtSetSystemInformation(SystemMemoryListInformation, …)`
ファイルキャッシュ	`SetSystemFileCacheSize`
メモリ優先度	`SetProcessInformation(ProcessMemoryPriority)`
サービス	`EnumServicesStatusExW`、`ControlService`、`ChangeServiceConfigW`、`EnumDependentServicesW`
特権	`OpenProcessToken`、`LookupPrivilegeValueW`、`AdjustTokenPrivileges`
全画面判定（GUI側）	`SHQueryUserNotificationState`、`GetLastInputInfo`
ウィンドウ終了（GUI側）	`EnumWindows`、`GetWindowThreadProcessId`、`PostMessageW(WM_CLOSE)`

参考になる既存実装として、調査メモにあった WinMemClean（多種の解放処理）と memclean（ゲーム中の保護）のソースは、API呼び出しの確認用に読む価値があります。ただし規模の小さいリポジトリなので、コードは流用前に内容を確認してください。

10. 開発ロードマップとテスト

最初は読み取り専用の可視化から作り、「何がメモリを使っているか」が正しく見えることを確かめてから、解放・常駐・停止の順に権限の要る機能を足します。



開発フェーズ · 5段階

各フェーズは、下の完了条件を満たしてから次に進みます。

フェーズごとの完了条件

1. **P1 監視のみ**: 機能単位のプライベートバイト合計が、タスクマネージャー「詳細」タブのコミットサイズ合計とおおむね一致する。Chrome・Edge・svchost・WSLが正しくまとまる。
2. **P2 手動解放**: L1〜L3が管理者権限で動作し、実行前後と60秒後の値が履歴に残る。保護リストのプロセスが一度も対象にならない。
3. **P3 常駐化**: サービスとトレイが分離し、トレイは一般権限で動く。24時間の連続稼働で常駐負荷が目標内（3章）に収まる。全画面中に自動解放が走らない。
4. **P4 機能停止**: すべての停止手段で、実行 → 再起動 → 復元 が元の状態に戻る。保護分類の機能には停止ボタンが出ない。
5. **P5 自動調整**: 人工的に逆効果を起こしたとき、クールダウン延長とレベル制限が働く。コード署名済みのインストーラー（MSIまたはMSIX）を配布できる。

テスト方針

- **単体テスト**: `policy` と `grouping` は入力のスナップショット（JSON）から判定結果を検証する。ヒステリシス、クールダウン、抑止条件、保護分類の境界値を網羅する。
- **保護リストの回帰テスト**: 組み込み保護プロセスと同名の偽プロセスを用意し、本物は保護され偽物は保護されないことを確認する。

- **結合テスト**: Windows 10とWindows 11のVM（Hyper-Vのチェックポイントから毎回復元）で、解放・停止・復元を一通り実行する。
- **メモリ圧迫の再現**: メモリを確保し続けるテスト用プログラムを用意し、トリガー発動と抑止が設計どおりに動くか確かめる。
- **効果の検証**: 同じ操作（アプリ切替え、ブラウザのタブ復帰など）の応答時間を、解放なし・L1・L2・L3で比較する。空きメモリではなく体感速度を評価の軸にする。

未決事項

- GUIフレームワークを egui と GPUI のどちらにするか（軽さならegui、見た目と滑らかさならGPUI）。
- ブラウザのタブ単位の表示まで行うか（拡張機能が必要になるため、初版ではブラウザ単位に留める案）。
- 配布形式（MSI / MSIX / ポータブル）とコード署名証明書の調達。